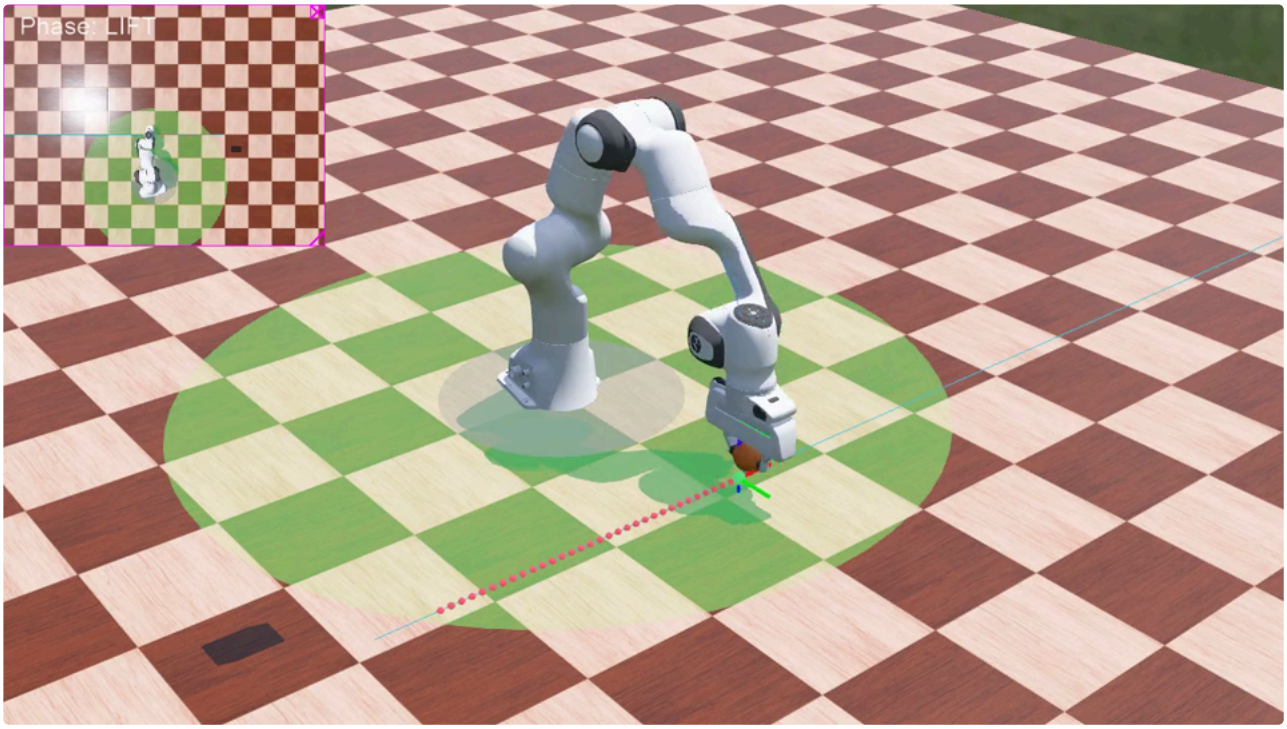
WEBOTS TEACHING EXAMPLE · USER GUIDE

Dynamic Grasping Lab

Catching a moving ball with the Franka Emika Panda — a hands-on robotics course in one simulation



A ball slides across the floor. A 7-joint robot arm **sees** it with a ceiling camera, **predicts** where it is going, **decides** where and when to meet it, **moves** there in time and **closes its gripper** at the right instant. Every one of those steps is a classic robotics building block, and this guide shows you how to explore each of them, step by step.

Simulator: Webots R2025a · Language: Python 3 + NumPy · Tested on Ubuntu 22.04
Based on *Dynamic Object Grasping Using a 7-DoF Manipulator* (MuJoCo) by Alaa Hussein and Haidar Saad, Manara University, supervised by Dr. Essa Alghannam.
Robot meshes and Panda hand: Webots Franka Emika Panda model (Apache 2.0).

Contents

1 What you will learn
2 Installing
3 Your first catch
4 A tour of the lab window
5 Teaching mode: one phase at a time
6 The seven phases, explained
7 Experiments to try
8 Programming exercises
9 For teachers: demos, recordings, data
10 Checking that everything works
11 Troubleshooting
12 Where things are in the code

How to read this guide

Chapters 2–4 get you running in about 15 minutes. Chapters 5–7 are the course: work through them with the simulation open. Chapter 8 is for learners who want to write code, chapter 9 for teachers, and chapters 10–12 are reference.

> **Tip.** Keep this guide on one side of the screen and the simulation on the other. Every "Try" box is meant to be done, not just read.

CHAPTER 1

What you will learn

Catching a moving object is harder than ordinary pick-and-place, because *timing* matters: the robot must arrive before the ball does and close its fingers at exactly the right moment. To do that, it chains seven steps, shown as the pipeline at the top of the lab window:

1 Observe camera + fit	2 Plan where & when	3 Approach quintic motion	4 Descend correct, go down	5 Wait timed closure	6 Close friction grasp	7 Lift check

Topic	What you will see and understand
Camera perception	Colour segmentation, blob analysis and the pinhole camera model turn an image into a ball position on the floor.
State estimation	A least-squares line fit turns noisy positions into a velocity; tracking and *gating* keep the estimate fresh while the arm moves.
Kinematics	Forward kinematics with screw theory (Product of Exponentials) and inverse kinematics by damped least squares.
Planning	Intercept planning: for each possible meeting time, is it reachable, does IK work, is the arm fast enough?
Trajectories	Quintic (minimum-jerk) motions and the shortest duration allowed by the joint speed limits.
Control	Velocity feed-forward plus proportional feedback for each joint.
Grasping	Friction, the minimum squeeze force $N_{min} = m \cdot g / (2\mu)$, and force-limited closing.

One method per step, on purpose. The example uses one planner, one trajectory type and one controller, chosen because they are the easiest to understand. There are no benchmarks or comparisons: the goal is to see each idea working inside a complete system.

What you need to know beforehand

- Basic Python (functions, lists, NumPy arrays) for the exercises.
- Vectors and matrices (rotation matrices help for kinematics).
- No prior Webots experience is needed.

CHAPTER 2

Installing

2.1 Webots R2025a

Download Webots R2025a from `cyberbotics.com` (free and open source). On Ubuntu, the Debian package installs it in one command:

```
wget https://github.com/cyberbotics/webots/releases/download/R2025a/webots_2025a_amd64.deb
sudo apt install ./webots_2025a_amd64.deb
```

Webots is about 160 MB to download and about 470 MB once installed. On Windows and macOS, use the installer from the website. The example is written for any platform but has been tested on Ubuntu 22.04.

2.2 Python and NumPy

The robot's program (its *controller*) is written in Python 3 and needs only NumPy:

```
pip install numpy
```

Watch out. Webots must use a Python that has NumPy. If several Pythons are installed, set the right one in Webots: *Tools → Preferences → Python command*.

2.3 The project

Download or clone the repository and keep the folder structure as it is:

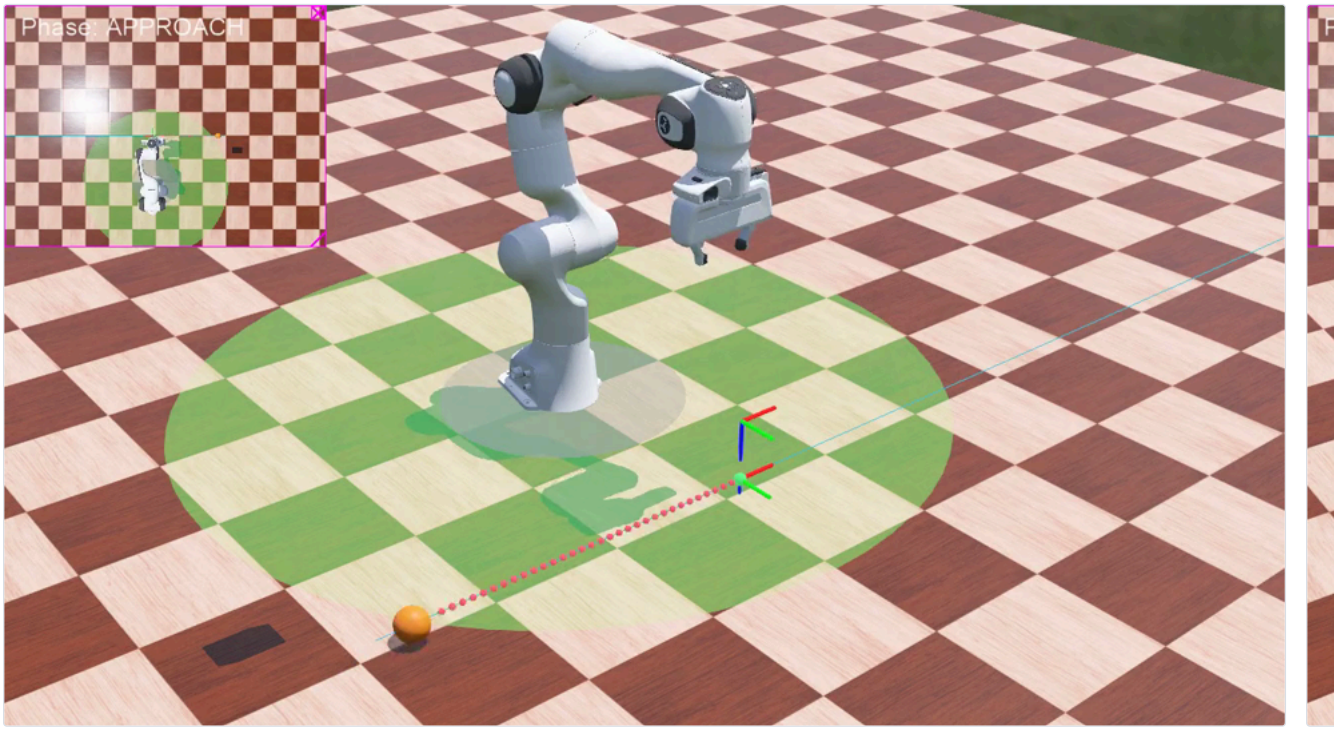
```
git clone https://github.com/haidar996/webots-dynamic-grasping.git
cd webots-dynamic-grasping
```

The robot's 3D model files are included, so the simulation also works offline. The first time you open the world, Webots may download the floor and sky textures; without internet it uses plain colours.

CHAPTER 3

Your first catch

1. **Open the world.** In Webots: *File* → *Open World…* and choose `worlds/dynamic_grasping.wbt`. (The first time, Webots shows a welcome window; click *Start Webots with the selected theme*.)
2. **Run it.** Press ▶ in the toolbar. The arm moves to its ready pose in about 3 seconds.
3. **Open the lab window. Double-click the robot arm** (or right-click it → *Show Robot Window*). On Linux it opens in your web browser. Its top-right corner should say **connected**.
4. **Launch a ball.** In the lab window, choose the scenario *Still ball* and press **▶ Launch ball**. Watch the robot pick it up.
5. **Go again.** Press **⟲ Reset**, wait until the arm is home (*Launch ball* becomes blue again), choose *Gentle roll* and launch.



Left: the ball (orange) slides along the predicted path; red dots are meeting points that were too late, the green dot is the chosen one; coloured axes show the target poses; the green disc is where the robot can grasp. Top left: what the ceiling camera sees. Right: caught and lifted.

Suggested order. *Still ball* → *Gentle roll* → *Original MuJoCo scene* → *Diagonal crossing* → *Coming towards the robot* → *Fast pass* → *Out of reach* (where the robot correctly refuses).

CHAPTER 4

A tour of the lab window

The left column holds the controls; the right side holds eight tabs. The bar at the top shows the seven phases: the current phase is blue, finished phases are green.

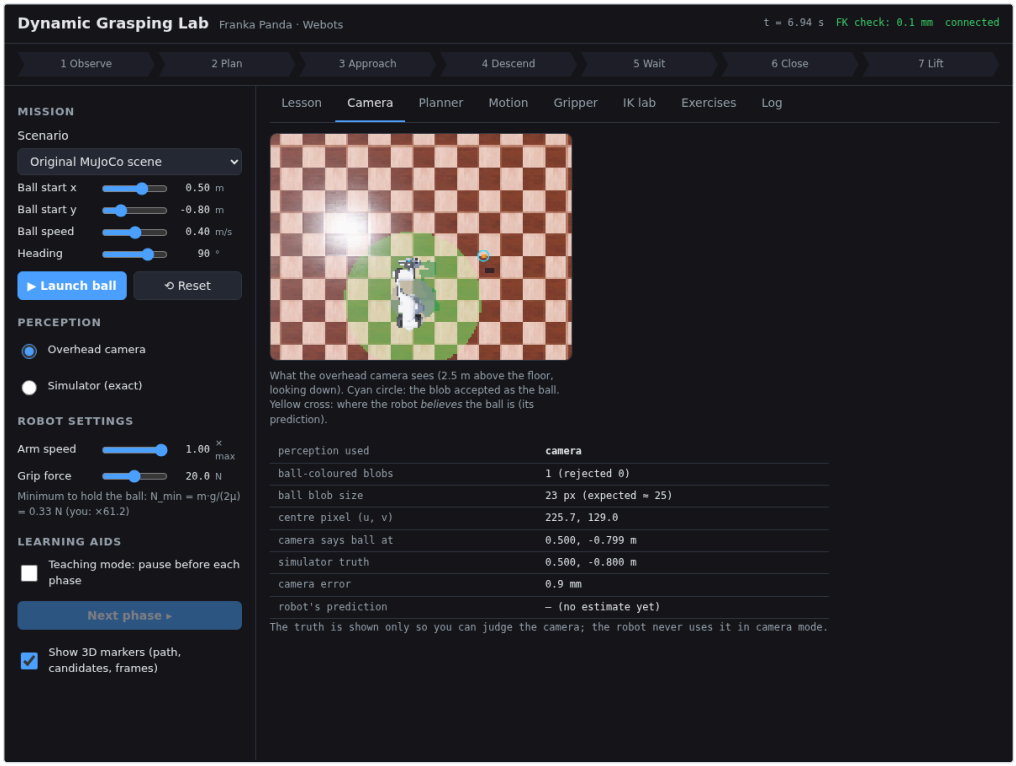
4.1 Controls (left column)

Control	What it does
Scenario	Seven ready-made launches. Choosing one fills in the four sliders below it.
Ball start x, y · speed · heading	Design your own launch. Heading is the direction of travel in degrees (90° = towards +y).
Launch ball / Reset	*Launch* only works when the arm is home (it is greyed out otherwise). *Reset* opens the gripper, puts the ball back and sends the arm home.
Perception	*Overhead camera*: the robot finds the ball in camera images. *Simulator (exact)*: perfect positions, no camera.
Arm speed	Fraction of the joint speed limits (0.3–1). Slower arm → later meeting point.
Grip force	Squeeze force per finger. The hint below shows the theoretical minimum, and turns orange below it.
Teaching mode	Pause before each phase (chapter 5). **Next phase ▸** continues.
Show 3D markers	Turns the predicted path, candidate dots and target frames on or off in the 3D view.

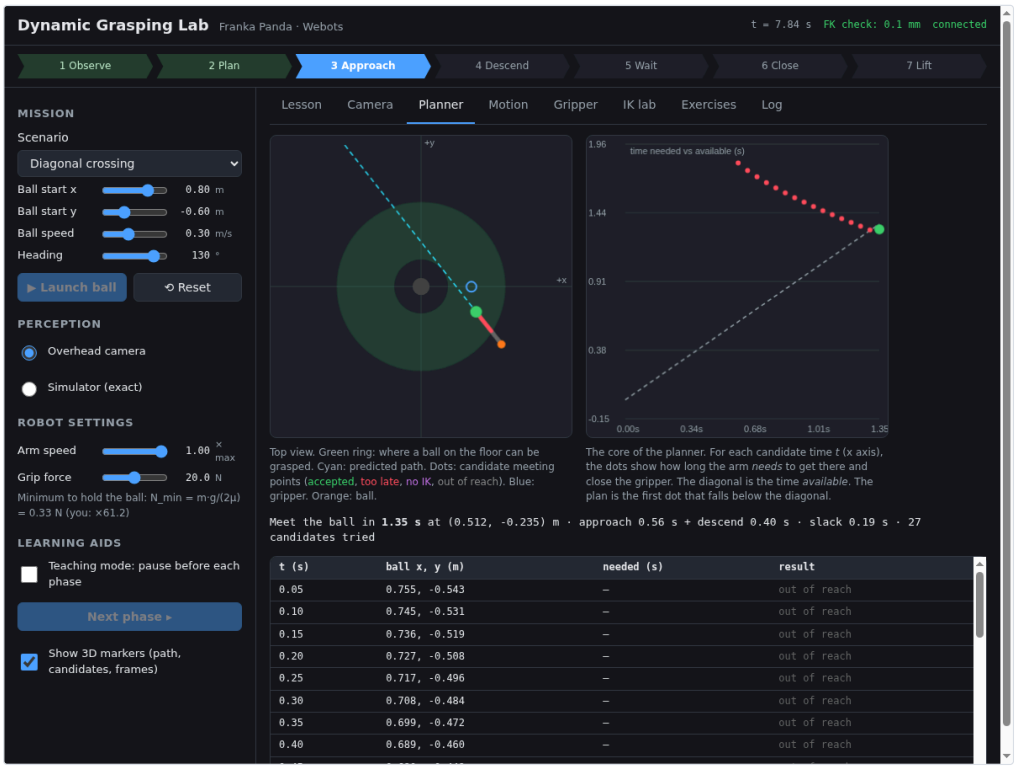
4.2 The tabs

Tab	What you find there
Lesson	A card for the current phase: the idea, the equations, the file/function that implements it, and a **Check yourself** quiz.
Camera	What the ceiling camera sees, the blob accepted as the ball (cyan circle), the robot's prediction (yellow cross), and the camera's error against the simulator's truth.
Planner	Top view with every candidate meeting point, the *time needed vs time available* chart, and a table of candidates with the reason each was rejected.
Motion	Desired (quintic) and measured angle of a chosen joint, the tracking error, and where each joint sits within its limits.
Gripper	Finger opening and finger force: watch the switch from moving to squeezing.
IK lab	Pick any gripper pose with sliders; inverse kinematics finds the joint angles and the arm moves there.
Exercises	The six programming exercises and a **Check my exercises** button (chapter 8).
Log	Everything the robot decided, with times, plus **CSV export** of the run data and planner candidates.

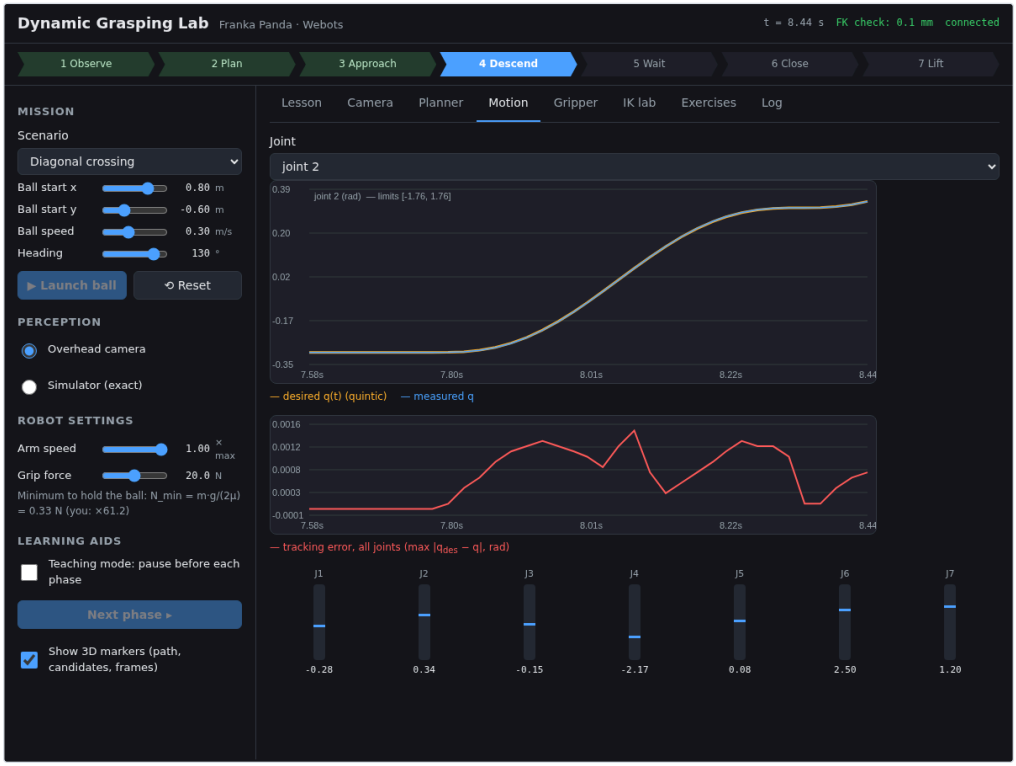
4.3 The tabs in pictures



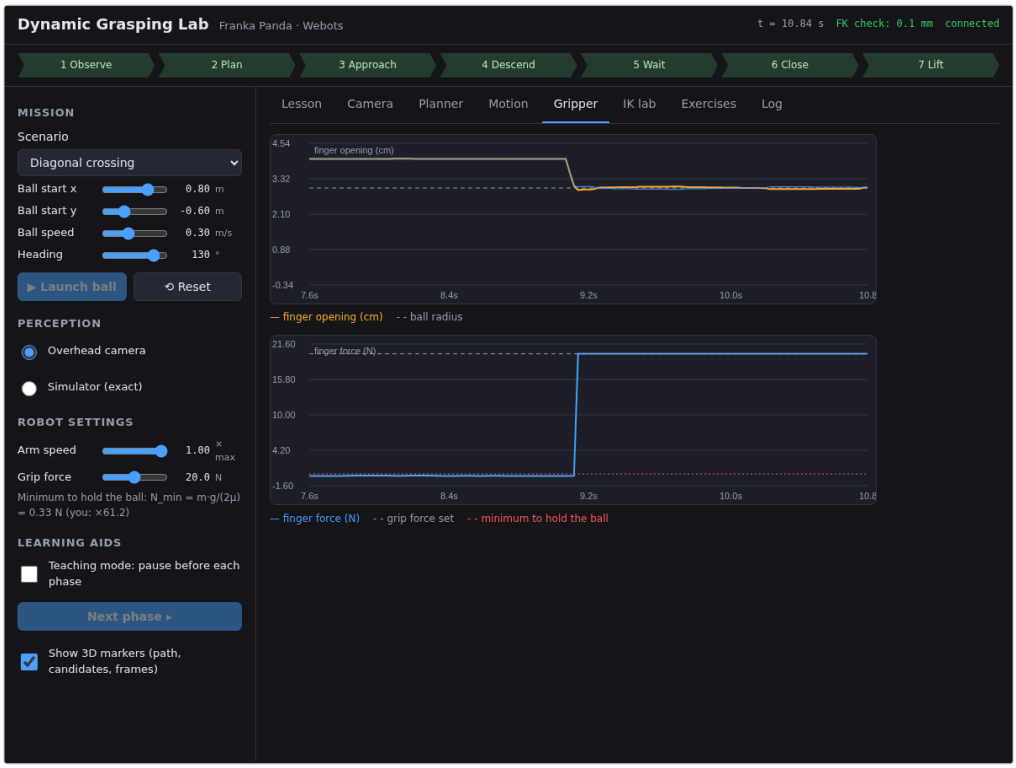
Camera tab: the ball is found 0.9 mm from its true position. The table explains each step of the detection.



Planner tab: candidate meeting points on the map, and the chart where "time needed" first drops below "time available".



Motion tab: the measured joint angle (blue) follows the quintic reference (orange) within a few milliradians.



Gripper tab: the fingers stop at the ball's radius (3 cm) and the force rises to the grip force you set.

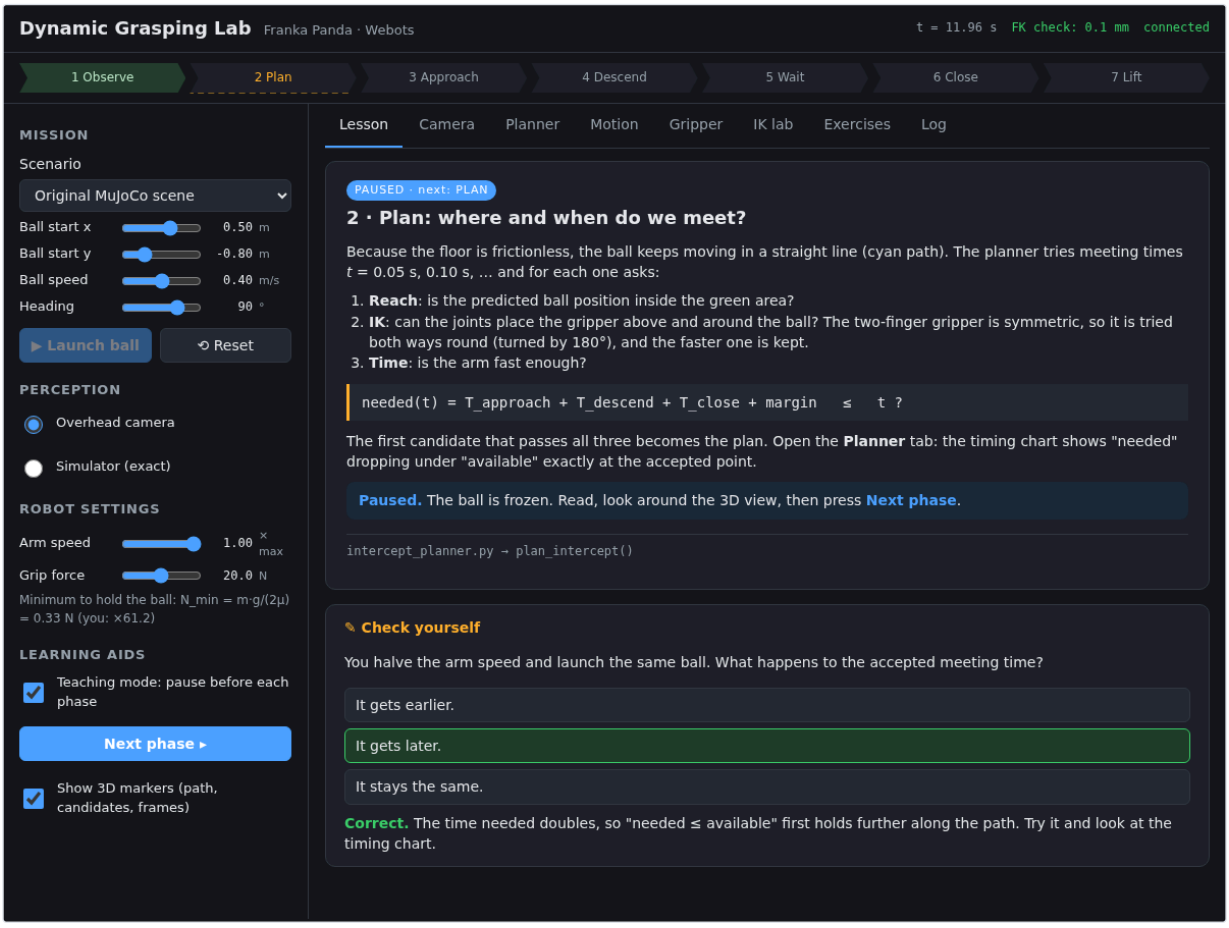
CHAPTER 5

Teaching mode: one phase at a time

Real catches take about two seconds, which is too fast to follow. Teaching mode stops the simulation logic before each phase, **freezes the ball**, and shows the lesson card for the phase that comes next.

1. Press ⟲ Reset and wait until the arm is home.
2. Tick **Teaching mode: pause before each phase**.
3. Choose *Original MuJoCo scene* and press ▶ Launch ball.
4. At each pause, read the **Lesson** card, answer the **Check yourself** quiz, look at the 3D view and the other tabs, then press Next phase ▸.

The robot pauses five times: before PLAN, APPROACH, DESCEND, WAIT and LIFT. Nothing is lost while paused: the plan and the predictions are shifted in time along with the frozen ball, so the catch still works after a long pause.



Paused before PLAN: the pipeline outlines the next phase, the lesson explains it, and the quiz has just been answered correctly.

CHAPTER 6

The seven phases, explained

1 · Observe: find the ball and estimate its motion

A 320×240 camera 2.5 m above the floor looks straight down. For each image: **segmentation** keeps orange pixels; **blob analysis** keeps the group of pixels with the size and shape of the ball; **back-projection** turns its centre pixel into a floor position:

```
f = (W/2) / tan(fov/2)                  focal length in pixels
ray = R · ( f, −(u − W/2), −(v − H/2) )  viewing ray in the world
p = c + t · ray,  with t chosen so that p is at the ball's height
```

Over 0.2 s (13 frames) a straight line is fitted by least squares; its slope is the velocity:

```
p(t) = p₀ + v · t        →   solve for p₀ and v
```

2 · Plan: where and when do we meet?

The floor is frictionless, so the ball goes straight. The planner tries meeting times t = 0.05, 0.10, … s and keeps the first one that passes three checks: the point is **reachable**; **IK** finds joint angles above and around the ball (trying the symmetric gripper both ways round); and the arm is **fast enough**:

```
needed(t) = T_approach + T_descend + T_close + margin   ≤   t
```

3 · Approach: a smooth joint-space motion

All seven joints follow a quintic from start to goal, with zero velocity and acceleration at both ends. The duration is the shortest that respects every joint's speed limit:

```
s(τ) = 10τ³ − 15τ⁴ + 6τ⁵,  τ = t/T       q(t) = q₀ + s(τ)·(q₁ − q₀)
T = max_j 15·|Δq_j| / (8·dq_max,j)
dq_cmd = dq_desired + K_p · (q_desired − q_measured)      (joint control)
```

Meanwhile the camera keeps **tracking** the ball: every new frame refreshes the line fit, and frames far from the prediction are ignored (*gating*).

4 · Descend: correct, then go down

The meeting point is **corrected** with the latest estimate (typically 1–20 mm), then the gripper goes straight down from 12 cm above, fingers open on either side of the ball's path. The gripper's x-axis points along the ball's motion, so the fingers form a gate.

5 · Wait: close at the right instant

```
close when  time_to_arrival ≤ (4 cm − 3 cm) / 0.2 m/s = 0.05 s
```

Near the gripper the hand hides the ball, so the last estimate is extrapolated for these final moments.

6 · Close: how hard to squeeze?

```
2 · μ · N ≥ m · g      →      N_min = m·g / (2μ) = 0.1 × 9.81 / 3 ≈ 0.33 N
```

The finger target is set inside the ball with the motor force capped at the grip force: the fingers move freely, touch, then squeeze at the cap. The two fingers are kept symmetric in software, as on the real Franka Hand.

7 · Lift and check

The gripper rises to 25 cm; the robot checks that the ball came with it. Lifting accelerates the ball, so the fingers must carry more than its weight, which is why the default grip force has a large safety factor.

CHAPTER 7

Experiments to try

Each experiment changes one thing. Predict the result first, then run it.

Arm speed. Launch *Original MuJoCo scene* at arm speed 1.0, then 0.5. In the Planner tab, where does the accepted point move, and why? (Answer: later along the path, because "time needed" doubles.)

Grip force. With *Still ball*, try 20 N, 2 N, 1 N and 0.2 N. At 0.2 N (below N_{min}) the ball slips during the lift; at 1 N it holds.

Camera vs simulator. Launch *Diagonal crossing* with each perception setting. Compare the "position error" and "Correction before descending" lines in the Log.

Out of reach. Launch the *Out of reach* scenario. Every candidate is grey or red in the Planner: a planner that says "impossible" quickly is as useful as one that finds a plan.

Faster and faster. Use *Fast pass* and raise the ball speed in steps of 0.1 m/s. When does the robot start to miss? Use the Log to explain why.

IK lab. Move the target to x = 0.8, z = 0.03. Does IK succeed? Compare with the green disc on the floor. Then change only the yaw by 180°: same point, different wrist angle.

Your own data. After a catch, export the run data (Log tab → *Export run data (CSV)*) and plot joint 2's desired and measured angle in Python or a spreadsheet.

More lessons, questions and extension projects are in `docs/LESSONS.md`.

CHAPTER 8

Programming exercises

Six short exercises in the `exercises/` folder, one per building block. Each file contains one function to write, with the formulas you need above it.

#	File	You write
1	`ex1_forward_kinematics.py`	$T(q) = e^{[S_1]q_1} \cdots e^{[S_7]q_7} M$
2	`ex2_quintic.py`	quintic position and shortest duration
3	`ex3_ik_step.py`	one damped-least-squares IK step
4	`ex4_velocity_fit.py`	least-squares line through noisy positions
5	`ex5_grip_force.py`	minimum grip force, with and without lifting
6	`ex6_pixel_to_floor.py`	pinhole back-projection, pixel to floor

1. Open an exercise file and replace `raise NotImplementedError` with your code.
2. Save, then press **Check my exercises** in the Exercises tab. There's no need to restart Webots. Or, in a terminal from the project folder: `python3 exercises/check.py` (add a number to check one exercise).
3. A wrong answer comes with a hint. When everything is correct the tab shows **6 / 6 exercises correct**.

```
$ python3 exercises/check.py
✔ Exercise 1: Forward kinematics (PoE)              Correct!
✘ Exercise 2: Quintic trajectory                    quintic_position is wrong at t = 0.21 s.
                                                    Check τ = t/T and s = 10τ³ − 15τ⁴ + 6τ⁵.
· Exercise 3: One IK step (damped least squares)    Not started: open exercises/ex3_ik_step.py
```

How the checker works. It compares your function with the code the robot actually runs, on many random inputs. A pass means your version would work in the simulation too. Each exercise file names the robot function that solves it, if you want to compare afterwards.

CHAPTER 9

For teachers: demos, recordings, data

9.1 Controller arguments

In the Webots scene tree, select the robot and edit its `controllerArgs` field:

Argument	Effect
`--launch=0.5,-0.8,0.4,90`	Launch this ball (x, y, speed, heading) as soon as the arm is home.
`--perception=simulator`	Start with exact positions instead of the camera.
`--grip-force=5`	Starting grip force in newtons.
`--script=file.json`	Play a list of window commands: a hands-free classroom demo.
`--record-movie=demo.mp4`	Record the 3D view from the launch to the result.
`--exit-on-result`	Quit Webots when the catch ends (exit code 0 = caught).

9.2 A scripted demo

A script is a JSON list of steps: wait for a phase, optionally hold it, then send a command, as if a user had pressed the button. This one launches a ball in teaching mode and moves on every 5 seconds:

```
[
  {"wait": "IDLE", "send": {"cmd": "teach", "on": true}},
  {"wait": "IDLE", "send": {"cmd": "launch", "x": 0.5, "y": -0.8, "speed": 0.4, "heading": 90}},
  {"wait": "PAUSED:PLAN",     "delay": 5, "send": {"cmd": "next"}},
  {"wait": "PAUSED:APPROACH", "delay": 5, "send": {"cmd": "next"}},
  {"wait": "PAUSED:DESCEND",  "delay": 5, "send": {"cmd": "next"}},
  {"wait": "PAUSED:WAIT",     "delay": 5, "send": {"cmd": "next"}},
  {"wait": "PAUSED:LIFT",     "delay": 5, "send": {"cmd": "next"}},
  {"wait": "CAUGHT"}
]
```

Commands include `launch`, `reset`, `teach`, `next`, `grip_force`, `arm_speed`, `perception`, `markers`, `ik_goal` and `check_exercises`. A full example covering every feature: `tests/webots_scripts/full_tour.json`.

9.3 Everything in one place

All tunable numbers (ball size, gains, camera period, safety margin, grip force...) are in `controllers/dynamic_grasping/config.py`. Change one, restart the simulation (Ctrl + Shift + T, *reset simulation*) and observe.

CHAPTER 10

Checking that everything works

10.1 Offline tests (no Webots needed)

```
pip install pytest
python3 -m pytest tests -q          # 20 tests
```

They check the kinematics against Franka's published DH table, inverse kinematics, the quintic limits, the planner, the camera geometry, the exercise checker, and the robot's whole decision logic run against a small stand-in for Webots.

10.2 Full tests in real Webots (no clicking needed)

```
pip install websockets
tests/run_webots_tests.sh           # about 8–10 minutes
```

This runs every scenario, a 46-step tour of every feature, and an end-to-end check of the lab window (headless Chrome clicks through it and reports any JavaScript error). It ends with **ALL PASSED**. While it runs, minimized Webots windows come and go: do not close them.

> **Last verified result** (Webots R2025a, Ubuntu 22.04): 20/20 offline tests; 7/7 scenarios; all 46 tour steps; 44/44 window checks; no errors or warnings.

CHAPTER 11

Troubleshooting

Problem	What to do
The lab window stays at "connecting..."	Press ▶ in Webots (the simulation must run), then reload the page (F5).
I cannot find the lab window	Double-click the robot arm, or right-click it → *Show Robot Window*. On Linux it opens in your browser, not inside Webots.
Launch ball is greyed out	The arm is not home yet. Press *Reset* and wait about 2 seconds.
"No module named numpy" in the Webots console	Install NumPy for the Python that Webots uses, or set it in *Tools* → *Preferences* → *Python command*.
The robot is invisible or grey	The 3D model files are in `protos/meshes/` ; keep the folder structure unchanged.
Floor or sky look plain	Webots could not download textures (no internet). Harmless.
"The camera could not see the ball"	The ball started outside the camera image or under the arm. Start it further away, or switch Perception to *Simulator*.
"No feasible intercept"	The ball never passes where the arm can be in time. Aim it through the green disc, slow it down, or raise the arm speed.
"The ball slipped out"	Grip force too low. Raise it well above the minimum shown under the slider.
Changes to the code have no effect	Restart the controller: reset the simulation (Ctrl + Shift + T) or reload the world.

CHAPTER 12

Where things are in the code

All robot code is in `controllers/dynamic_grasping/`. Each file is one idea and can be read on its own.

File	Contents
`dynamic_grasping.py`	The state machine (the seven phases), joint control, tracking, messages to the window.
`config.py`	Every tunable number in one place.
`perception.py`	Camera: segmentation, blob analysis, back-projection.
`kinematics.py`	Product-of-Exponentials forward kinematics, Jacobian, damped-least-squares IK.
`trajectory.py`	Quintic trajectories and their minimum duration.
`intercept_planner.py`	The forward-scan intercept search.
`gripper.py`	Finger control, grip force, finger coupling.
`scene_markers.py`	Teaching markers drawn in the 3D view.
`command_script.py`	Plays scripted window commands.

Elsewhere	Contents
`worlds/`	The scene: arm with ceiling camera, ball, frictionless floor.
`protos/`	The robot model (`EduPanda.proto`), the Panda hand and the meshes.
`plugins/robot_windows/`	The lab window (HTML, CSS, JavaScript).
`exercises/`	The six exercises and their checker.
`tests/`	Offline tests, the Webots test runner, the window check.
`docs/LESSONS.md`	The full lessons with questions and extension projects.
`PORTING_NOTES.md`	What changed from the MuJoCo original, and what testing in Webots taught us.

Credits. Based on *Dynamic Object Grasping Using a 7-DoF Manipulator* (MuJoCo) by Alaa Hussein and Haidar Saad, Manara University, supervised by Dr. Essa Alghannam. Robot meshes and the Panda hand come from the Webots Franka Emika Panda model (Apache 2.0).